

COURS PHP

Fonctions & Chaînes de caractères

Module 3 & Module 4 — Guide complet avec exemples

Fonctions nommées

Fonctions anonymes

Chaînes de caractères

Les Fonctions en PHP

Organiser, réutiliser et structurer son code



Qu'est-ce qu'une fonction ?



Analogie : une fonction = une machine à café. Tu donnes du café et de l'eau (paramètres), tu appuies sur le bouton → tu reçois du café (résultat). Pas besoin de savoir comment ça marche à l'intérieur !



Définition

Un bloc de code nommé, exécutable autant de fois qu'on veut depuis n'importe où dans le programme.



Avantages

- ✓ Réutilisation du code
- ✓ Lisibilité améliorée
- ✓ Maintenance facilitée
- ✓ Débogage simplifié



Types en PHP

- Natives : strlen(), array_push()...
- Personnalisées : créées par le dev
- Anonymes & fléchées (voir plus loin)



```
<?php
function direBonjour() {
    echo "Bonjour tout le monde !";
}

// Appel (peut être répété n fois)
direBonjour(); // Bonjour tout le monde !
direBonjour(); // Bonjour tout le monde !
?>
```



Syntaxe d'une fonction PHP

```
<?php
// mot-clé      nom      paramètres
//      ↓           ↓           ↓
function additionner( $a,  $b ) {
    $resultat = $a + $b;    // corps
    return $resultat;      // valeur renvoyée
}

$somme = additionner(5, 3);
echo $somme;                // 8
echo additionner(10, 20); // 30
?>
```

function

Mot-clé obligatoire pour déclarer une fonction

nom()

Nom descriptif, unique, commence par lettre ou _

{ ... }

Accolades : corps de la fonction, exécuté à l'appel

return

Renvoie une valeur à l'appelant et arrête la fonction



Paramètres et Arguments

PARAMÈTRE

Variable dans la DÉCLARATION de la fonction.
C'est un espace réservé pour recevoir une valeur.

Ex : `function bonjour($prenom) { ... }`



ARGUMENT

Valeur réelle passée lors de l'APPEL.
C'est la vraie valeur qui remplace le paramètre.

Ex : `bonjour("Khadija");`

```
<?php
function bonjour($prenom) {      // $prenom = PARAMÈTRE
    echo "Bonjour " . $prenom . " !";
}

bonjour("Khadija"); // "Khadija" = ARGUMENT → Bonjour Khadija !
bonjour("Moussa");  //              Bonjour Moussa !

function infos($nom, $age, $ville) {
    echo "$nom a $age ans et vit à $ville.";
}
infos("Ali", 20, "Dakar"); // Ali a 20 ans et vit à Dakar.
?>
```

Paramètres Optionnels (Valeurs par défaut)

💡 Un paramètre optionnel a une valeur par défaut. Si l'appelant ne fournit pas l'argument, la valeur par défaut est utilisée automatiquement.

```
<?php
function calculerPrix($prix, $reduction = 0) { // $reduction optionnel
    return $prix - ($prix * $reduction / 100);
}

echo calculerPrix(100);           // 100  (pas de réduction)
echo calculerPrix(100, 20);      // 80   (20% de réduction)
echo calculerPrix(200, 50);      // 100  (50% de réduction)

// ⚠️ RÈGLE : les paramètres optionnels doivent être EN DERNIER
function ok($obligatoire, $opt = 0) { } // ✅ Correct
function err($opt = 0, $obligatoire) { } // ❌ Erreur PHP
?>
```

✓ Fonctions plus flexibles

✓ Évite de répéter les cas courants

✓ Rétrocompatibilité facile

Passage par valeur vs Passage par référence

PAR VALEUR (défaut)

La fonction reçoit une COPIE.
Modifier la copie n'affecte pas l'original.

```
<?php
function doubler($nb) {
    $nb = $nb * 2; // copie modifiée
}
$x = 5;
doubler($x);
echo $x; // → 5 (inchangé !)
?>
```

PAR RÉFÉRENCE (&)

La fonction reçoit l'ADRESSE.
Modifier à l'intérieur change l'original !

```
<?php
function doubler(&$nb) { // & = référence
    $nb = $nb * 2;
}
$x = 5;
doubler($x);
echo $x; // → 10 (modifié !)
?>
```

 Conseil : utilisez `return` pour renvoyer une valeur modifiée. La référence est puissante mais rend le code plus difficile à lire.



Typage des Paramètres (PHP 7+)

💡 PHP 7 introduit le typage strict : on peut préciser le TYPE attendu pour chaque paramètre et le type de retour. PHP génère une erreur si le type ne correspond pas.

```
<?php
declare(strict_types=1); // mode strict (en haut du fichier)

//          type param1  type param2  type retour
function additionner(int $a, int $b): int {
    return $a + $b;
}

echo additionner(5, 3); // ✓ 8
echo additionner(5.5, 3); // ✗ TypeError (mode strict)
echo additionner("5", 3); // ✗ TypeError (mode strict)

function estMajeur(int $age): bool {
    return $age >= 18;
}

// ?int = peut retourner null OU int
function trouver(string $nom): ?int { /* ... */ }
?>
```

int

float

string

bool

array

null

mixed

?type



Portée des Variables (Scope)

Variable LOCALE

Créée DANS une fonction.
N'existe que pendant son exécution.
Inaccessible depuis l'extérieur.

```
<?php
function maFonction() {
    $local = "Je suis local";
    echo $local; // ✓ OK
}
maFonction();
echo $local; // ✗ Erreur !
// $local n'existe pas ici
?>
```

Variable GLOBALE

Créée EN DEHORS des fonctions.
Utiliser dans une fonction : mot-clé global requis.

```
<?php
$compteur = 0; // GLOBALE

function incrementer() {
    global $compteur; // accès
    $compteur++;
}

incrementer(); echo $compteur; // 1
incrementer(); echo $compteur; // 2
?>
```

⚠ Évitez trop de variables globales — elles compliquent les tests. Préférez passer les valeurs en paramètres.



Superglobales & Variables Statiques

Superglobales

Accessibles partout, sans mot-clé global.

\$_GET

Données URL

\$_POST

Données
formulaire

\$_SESSION

Session

\$GLOBALS

Toutes
globales

\$_SERVER

Infos serveur

\$_COOKIE

Cookies

Variables Statiques

Conservent leur valeur entre les appels.

```
<?php
function compter() {
    static $nb = 0; // initialisé UNE FOIS
    $nb++;
    echo $nb . "\n";
}

compter(); // 1
compter(); // 2 (pas réinitialisé !)
compter(); // 3

// Sans static → toujours 1
function compterSans() {
    $nb = 0; // réinitialisé à chaque appel
    $nb++;
    echo $nb; // toujours 1 !
}
?>
```



Fermetures / Closures

💡 Une closure est une fonction anonyme qui peut capturer des variables extérieures grâce au mot-clé `use`. Elle les "enferme" dans son propre univers.

```
<?php
$message = "Bonjour"; $nom = "Khadija";

// Capture par COPIE avec use
$saluer = function() use ($message, $nom) {
    echo "$message, $nom !";
};
$saluer(); // Bonjour, Khadija !

// Capture par RÉFÉRENCE avec use (&$var)
$compteur = 0;
$inc = function() use (&$compteur) { $compteur++; };
$inc(); $inc();
echo $compteur; // 2

// Closure comme callback
$nombre = [5, 2, 8, 1];
usort($nombre, function($a, $b) { return $a - $b; });
print_r($nombre); // [1, 2, 5, 8]
?>
```

`use ($var)` →

Capture par copie — l'original n'est pas modifié

`use (&$var)` →

Capture par référence — modifie l'original

`callback` →

Parfait pour `usort()`, `array_map()`, `array_filter()`...



Fonctions Anonymes



Une fonction anonyme est une fonction SANS NOM. Elle peut être stockée dans une variable, passée en paramètre ou retournée par une autre fonction.



```
<?php
// 1. Stocker dans une variable
$carre = function($n) { return $n * $n; };
echo $carre(5); // 25

// 2. Passer comme callback
$nombre = [1, 2, 3, 4, 5];
$doubles = array_map(function($n) { return $n * 2; }, $nombre);
print_r($doubles); // [2, 4, 6, 8, 10]

// 3. Retourner une fonction depuis une fonction
function multiplicateur($facteur) {
    return function($n) use ($facteur) { return $n * $facteur; };
}
$fois3 = multiplicateur(3);
echo $fois3(5); // 15
?>
```



Anonyme : flexible, usage local, idéale comme callback



Classique : accessible partout, réutilisable, nommée

⚡ Fonctions Fléchées — Arrow Functions (PHP 7.4+)

💡 *Syntaxe raccourcie des fonctions anonymes. Elles capturent automatiquement les variables extérieures — pas besoin de use !*

```
<?php
// AVANT (anonyme classique)
$carre = function($n) { return $n * $n; };

// APRÈS (fléchée) – beaucoup plus courte !
$carre = fn($n) => $n * $n;
echo $carre(5); // 25

// Capture automatique des variables extérieures
$taxe = 0.2;
$prixTTC = fn($ht) => $ht * (1 + $taxe); // $taxe capturée auto !
echo $prixTTC(100); // 120.0

// Avec array_map
$nombre = [1, 2, 3, 4, 5];
$carres = array_map(fn($n) => $n * $n, $nombre);
print_r($carres); // [1, 4, 9, 16, 25]
?>
```

✅ Avantages : syntaxe concise, capture auto, très lisible

⚠️ Limites : une seule expression, pas de { }, capture en lecture seule

Les Chaînes de Caractères

Manipuler, transformer et explorer les textes en PHP



Les Chaînes de Caractères — Définition

Une chaîne (string) est une suite de caractères : lettres, chiffres, espaces, symboles. C'est l'un des types les plus utilisés en PHP.

```
<?php
// Guillemets doubles → variables et séquences interprétées
$prenom = "Khadija";
$msg1 = "Bonjour $prenom !"; // Bonjour Khadija !
$msg2 = "Ligne1\nLigne2";    // \n = saut de ligne

// Guillemets simples → tout est LITTÉRAL (plus rapide)
$msg3 = 'Bonjour $prenom !'; // Bonjour $prenom ! (non remplacé)
$msg4 = 'C\'est cool !';     // échapper l'apostrophe avec \

// Heredoc → comme guillemets doubles, sur plusieurs lignes
$texte = <<<EOT
Bonjour $prenom,
Comment allez-vous ?
EOT;

// Nowdoc → comme guillemets simples, sur plusieurs lignes
$texte2 = <<<'EOT'
Bonjour $prenom, // pas interprété
EOT;
?>
```

"..."

Variables
interprétées

'...'

Tout littéral, rapide

<<<EOT

Heredoc
(multiligne)

<<<'EOT'

Nowdoc (littéral)



Fonctions de Manipulation — Les Essentielles

strlen(\$str)

Longueur

```
strlen("Bonjour") → 7
```

strtoupper(\$str)

En majuscules

```
strtoupper("bonjour") →  
"BONJOUR"
```

str_replace(\$r,\$n,\$s)

Remplacer dans chaîne

```
str_replace("o","0","Bonjo  
ur") → "B0njour"
```

substr(\$str,\$s,\$len)

Extraire sous-chaîne

```
substr("Bonjour",3,4) →  
"jour"
```

explode(\$sep,\$str)

Chaîne → tableau

```
explode(",","a,b,c") →  
["a","b","c"]
```

sprintf(\$fmt,...)

Formatage avancé

```
sprintf("%.2f €", 9.5) →  
"9.50 €"
```

strtolower(\$str)

En minuscules

```
strtolower("BONJOUR") →  
"bonjour"
```

trim(\$str)

Supprimer espaces bord

```
trim(" salut ") →  
"salut"
```

strpos(\$str,\$find)

Position sous-chaîne

```
strpos("Bonjour","jour") →  
3
```

str_contains(\$s,\$sub)

Vérifie si contient (PHP 8)

```
str_contains("Bonjour","jo  
ur") → true
```

implode(\$sep,\$arr)

Tableau → chaîne

```
implode("-",["a","b"]) →  
"a-b"
```

ucfirst(\$str)

1ère lettre majuscule

```
ucfirst("bonjour") →  
"Bonjour"
```




Exemples Pratiques

```
<?php
// 1. Nettoyer et valider une saisie email
function nettoyerEmail(string $email): string {
    $email = trim($email);          // supprimer espaces
    $email = strtolower($email);    // tout en minuscules
    return filter_var($email, FILTER_SANITIZE_EMAIL);
}
echo nettoyerEmail(" KHADIJA@Example.COM "); // khadija@example.com

// 2. Compter les mots d'un texte
$texte = "PHP est un langage de programmation puissant";
$mots = explode(" ", $texte);
echo count($mots) . " mots"; // 7 mots

// 3. Vérifier si un mot existe
if (str_contains($texte, "PHP")) {
    echo "PHP est mentionné !";
}

// 4. Formater un prix
$prix = 15.5;
echo sprintf("Prix : %.2f €", $prix); // Prix : 15.50 €
?>
```



RÉCAPITULATIF COMPLET



Fonction nommée

```
function nom($p) { return ...; }  
Réutilisable partout dans le  
code.
```



Paramètres

```
Défaut : $p = 0  
Référence : &$p  
Typage PHP 7+ : int $p
```



Portée

```
Locale : dans la fonction  
Globale : global $var  
Statique : static $var
```



Fonctions anon.

```
$f = function($n) { ... };  
Capture : use ($var)  
Callback pour array_map...
```



Arrow fn (7.4+)

```
$f = fn($n) => $n * 2;  
Capture auto, 1 expression  
Pas besoin de use
```



Chaînes

```
strlen, strtolower, trim  
str_replace, substr, strpos  
explode, implode, sprintf
```

